

Laboratory Work No. 4

Multiple Object Tracking

Student Name: [Your Name]

Group/ID: [Your Group]

Detector Used: [e.g., YOLO11n from Lab 3]

Trackers: BoT-SORT, ByteTrack

Instructor: Assoc. Prof. A. Kroshchanka

Date: September 8, 2026

Brest State Technical University
Department of Intellectual Information Technologies

Abstract

This report explores the application of multiple object tracking algorithms using a custom-trained object detector. Using the Ultralytics YOLO framework, tracking algorithms such as BoT-SORT and ByteTrack are implemented and applied to video sequences. The study investigates how modifying parameters within tracking configuration files affects track stability and identifier (ID) consistency. Tracking logic is evaluated using real-world videos sourced from the Internet that contain the objects classified during the previous laboratory work.

Contents

1	Introduction and Objectives	2
2	Methodology	2
2.1	Tracker Algorithms Overview	2
2.2	Implementation	2
3	Experimentation: Configuration Parameters	3
3.1	Observations on Parameter Changes	3
4	Results and Visualizations	3
5	Conclusion	4

1 Introduction and Objectives

Laboratory Goal

To investigate the application of tracking algorithms based on a trained object detector network.

The specific tasks to be completed in this laboratory work are:

1. Implement logic for tracking multiple objects using the Ultralytics YOLO library, based on the detector trained in Laboratory Work 3.
2. Apply BoT-SORT and ByteTrack algorithms using their respective configuration files.
3. Investigate the changes in parameters within the configuration files and their impact on tracking quality.
4. Utilize video materials from the Internet (e.g., YouTube) containing multiple instances of the classes from Lab 3 as source data for the experiments.
5. Format the report and upload the source code and report to the corresponding GitHub repository.

2 Methodology

2.1 Tracker Algorithms Overview

Multiple Object Tracking (MOT) extends standard object detection by assigning a persistent unique identifier (ID) to each detected object across consecutive video frames. In this laboratory work, two distinct tracking algorithms integrated into the Ultralytics framework are evaluated:

- **BoT-SORT**: A robust tracker that balances bounding box geometry with appearance features to maintain track identities even during occlusions.
- **ByteTrack**: An efficient tracking algorithm that leverages both high and low-confidence detection boxes to reduce false negatives and maintain trajectory consistency.

2.2 Implementation

The tracking logic utilizes the custom weights (`best.pt`) generated in Lab 3. The ‘track’ mode in Ultralytics YOLO allows direct tracking on video streams or URLs (such as YouTube videos).

```
1 from ultralytics import YOLO
2
3 # Load the custom model from Lab 3
4 model = YOLO('path/to/Lab3/runs/detect/train/weights/best.pt')
5
6 # Source video (e.g., YouTube link or local video file)
```

```
7 video_source = "https://www.youtube.com/watch?v=example"
8
9 # Run tracking using ByteTrack
10 results_byte = model.track(source=video_source, tracker="bytetrack.yaml",
11                             , show=True, save=True)
12
13 # Run tracking using BoT-SORT
14 results_bot = model.track(source=video_source, tracker="botsort.yaml",
15                             , show=True, save=True)
```

Listing 1: Basic Tracking Implementation with Ultralytics

3 Experimentation: Configuration Parameters

To investigate the effect of different tracking hyperparameters, custom YAML configuration files were created and modified. Key parameters analyzed include:

- `track_high_thresh`: The confidence threshold for the first association.
- `track_low_thresh`: The confidence threshold for the second association (ByteTrack specific).
- `match_thresh`: Threshold for matching detections to existing tracks.

```
1 # custom_tracker.yaml
2 tracker_type: bytetrack
3 track_high_thresh: 0.6 # Adjusted from default 0.5
4 track_low_thresh: 0.1
5 new_track_thresh: 0.7
6 track_buffer: 30 # Frames to keep lost tracks
7 match_thresh: 0.8
```

Listing 2: Customizing Tracker Configuration (YAML)

3.1 Observations on Parameter Changes

By adjusting the `track_buffer` and `match_thresh`, significant differences in tracking stability were observed. For instance, increasing the track buffer allowed the tracker to recover object IDs after brief occlusions, reducing the number of ID switches.

4 Results and Visualizations

The tracking algorithms were applied to video sequences containing the target objects. The outputs demonstrate bounding boxes overlaid with unique integer IDs that persist across frames.

The comparison between BoT-SORT and ByteTrack revealed that... (Add your specific findings regarding frame rate performance, ID switching, and robustness against occlusion here).

5 Conclusion

In this laboratory work, the custom object detector trained in Lab 3 was successfully extended to perform Multiple Object Tracking (MOT). By utilizing the Ultralytics YOLO framework, both BoT-SORT and ByteTrack algorithms were applied to real-world video sequences. The investigation into the tracking configuration parameters highlighted the importance of fine-tuning thresholds and buffers to minimize ID switches and improve trajectory continuity.

References

- [1] Ultralytics. (2024). *Multi-Object Tracking with Ultralytics YOLO*. Retrieved from <https://docs.ultralytics.com/modes/track/>